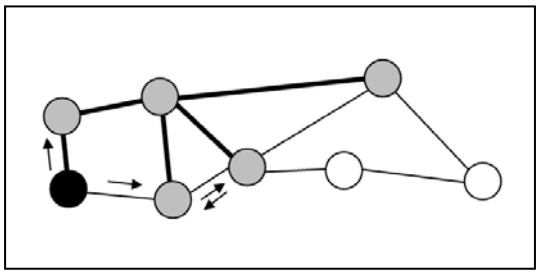
Prof. Dr. Erwin Mayer

Fakultät für Elektrotechnik
und Informationstechnik

Praktikum Verteilte Systeme

Versuch 3

Der Echo-Algorithmus im Einsatz



Was Sie in diesem Versuch lernen:

- Vertrautwerden mit verteilten Anwendung und Prinzipien
- Anwendung des Probe/Echo-Algorithmus in einem Verteilten System
- Durchführung einer verteilten Berechnung
- Aufbau einer Verteilten Test-/Simulationsumgebung
- Absprache von Schnittstellen im Team und verteilte Komponentenentwicklung
- Testen von verteilten Anwendungen

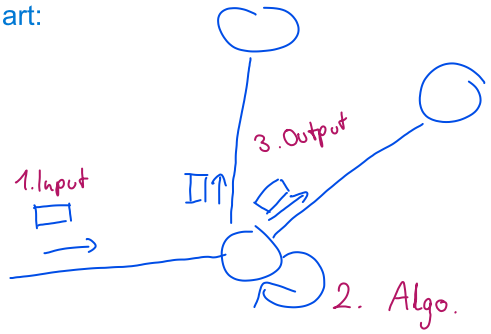
Voraussetzug zur Teilnahme: Pre-Lab Aufgaben komplett durchgeführt (mit Nachweis) !

1 PRE-LAB-Fragen

Name	Matrikelnummer	Semester

1. Welche Struktur ("Bauart") haben verteilte Algorithmen in der Regel? Geben Sie eine Skizze und die Beschreibung dazu. (Suchen Sie die passende Folie aus der Vorlesung!)

   Event/Action Bauart:

2. Welche allgemeinen Eigenschaften muss man für einen verteilten Algorithmus nachweisen?

   Korrektheitseigenschaft
   Terminierungseigenschaft
   Fairness
   Starvation-freeness
   Liveness - Eigenschaft

3. Angenommen Sie müssten auf einem einzelnen Rechner N Knoten simulieren, die miteinander Nachrichten austauschen. Wie würden Sie dies machen?

   Knoten = Threads / Prozesse

   Nachrichtenaustausch = UDP - Sockets

4. Wie/Wo würden Sie in diesem Fall die Latenz der Nachrichten simulieren?

   beim versende / empfangen mittels sleep

5. Geben Sie den in der Vorlesung vorgestellten Flooding-Algorithmus als Pseudo-Code wieder.

```
Sende Nachricht mit Inhalt I an alle direkten Nachbarn
informed = false;

alle:
  event: Nachricht von Nachbar N trifft ein
  action: If (not informed) {
            speicher Inhalt I;
            informed = true;
            Sende Nachricht I an alle Nachbarn außer N
```

6. Geben Sie den in der Vorlesung vorgestellten Echo-Algorithmus als Pseudo-Code wieder.

```
Zu beginn:
  alle knoten:
    informed = false
    neighbors_informed = 0

    event: nachbar mit inhalt i oder e von nachbar n trifft ein
    action: neighbors_informed++;
            if (not informed){
              informed = true;
              upward_node = N;
              sende Nachricht I an alle direkten Nachbarn außer N
            }
            if (neighbors_informed == anzahl direkter Nachrichten){
              if (initiator){
                print 'echo-algo hat terminiert'
              } else {
                sende e nachricht an upward_node
              }
            }
```

2 Praktikum

2.1 Allgemein

Der dritte Versuch im Praktikum soll Sie mit Verteilten Algorithmen vertraut machen und den in der Vorlesung vorgestellten Flooding/Echo-Algorithmus verwenden, um eine einfache verteilte Berechnung durchzuführen.

Optionale Versuchsteile können den entwickelten Algorithmus später kombinieren mit weitergehenden Algorithmen (Snapshot-Algorithmen, Election-Verfahren, etc.).

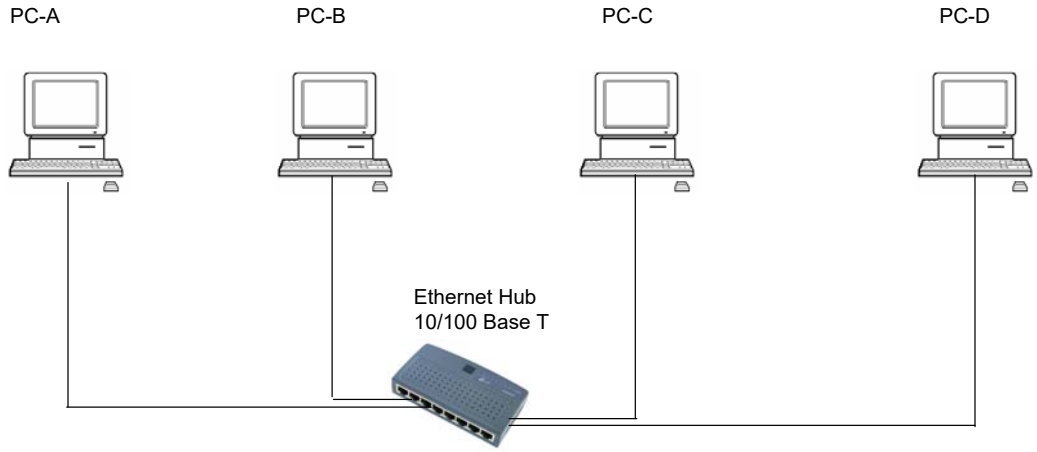
Da die Realisierung eines verteilten Algorithmus erst ab einer bestimmten Knoten- und Kantenanzahl eines Netzwerks interessant wird, beschäftigt sich ein Teil des Versuches mit der SIMULATION eines verteilten Systems und der anschließenden BEOBACHTUNG des Ablaufs. Hierzu wird eine eigene kleine Infrastruktur benötigt, welche Sie selbst entwickeln sollen.

Die Aufgabe selbst besteht darin, einen verteilten Algorithmus zur Summenbildung in einem beliebigen und vorher unbekannten Netzwerk durchzuführen. Er soll dazu dienen, die Gesamtmenge an verfügbarem Speicherplatz in einem Netzwerk zu ermitteln.

Jedem Knoten wird zunächst ein zufälliger numerischer Wert zugewiesen, dies kann z.B. der verfügbare Speicherplatz auf diesem Knoten in Gigabyte repräsentieren. Der Algorithmus soll nun in einem verteilten Ablauf ohne zentralen Koordinator und ohne Kenntnis der Topologie und Teilnehmerzahl, die Summe der Werte aller Knoten, d.h. die Gesamtmenge an Speicherplatz im Netzwerk berechnen.

2.2 Netzwerkinfrastruktur

Wie bei den Versuchen zu Computernetzen verwendet jede Gruppe eine Insel mit 4 Rechnern, die über einen Hub zu einem gemeinsamen Subnetz zusammengeschlossen sind.



2.3 Entwicklungsumgebung

Sie sollen folgende Entwicklungsumgebung einsetzen:

- LINUX
- JAVA
- UDP

2.4 Gruppenbildung

Sie arbeiten in Gruppen zu je maximal 3 Studierenden zusammen, welche eine gemeinsame verteilte Anwendung entwickeln. JEDER der 3 Teilnehmer muss dabei alle Komponenten programmieren.

Obwohl jeder seine Komponenten für sich entwickelt, arbeitet die Gruppe zusammen.

Erster Labortermin:

1. Die Gruppe überlegt sich zusammen einen verteilten Algorithmus zur Lösung der Aufgabe und skizziert diesen in Pseudocode.

2. Die Gruppe definiert gemeinsam die Schnittstellen zwischen den verteilten Komponenten (**denken Sie an ALLE Schnittstellen**typen, insbesondere

   → Format I/E-Nachrichten
   → Format Log-Nachrichten (s.u.)
   → Format Start/Result-Nachrichten (s.u.)
   → Format der Kommandozeilenparameter der zu startenden Prozesse (s.u.)
   (Achtung: bitte hier keine Semikolons im Format verwenden !
   später wichtig für ssh-Ausführung)

CHECKPOINT: Abnahme durch Laborleiter

3. Jeder implementiert eine Lösung und macht Tests auf seinem Entwicklungsrechner. (= Heimarbeit)

Zweiter Labortermin:

4. Die Gruppe entwickelt zusammen die Infrastruktur, um Komponenten unterschiedlicher Gruppenmitglieder **remote** zu starten

5. Gruppe testet zusammen die zuhause entwickelten Komponenten.

CHECKPOINT: Abnahme durch Laborleiter

6. Jeder implementiert eine kleine Erweiterung des Algorithmus (= Heimarbeit).

2.5 Aufgabenstellung

2.5.1 Teil 1: Verteilte Summenbildung

Implementieren Sie auf einem simulierten Netzwerk einen verteilten Algorithmus, der die Größe des insgesamt verfügbaren Speichers im Netzwerk (d.h. die Summe des Speichers auf allen Knoten) berechnet.

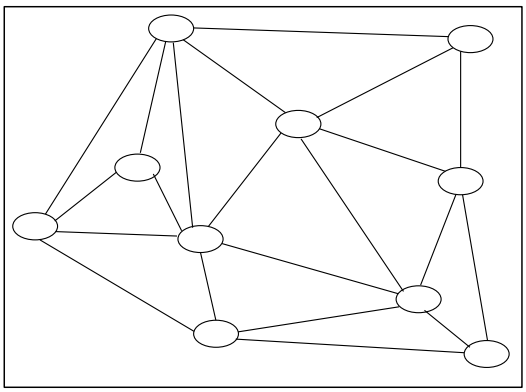


Tipp:

Nehmen Sie den in der Vorlesung vorgestellten Echo-Algorithmus als Grundlage und erweitern Sie diesen entsprechend.

2.5.2 Teil 2: Aufbau eines Simulationsumgebung

Zum Test Ihres Verteilten Algorithmus mit einer größeren Anzahl Knoten benötigen wir ein Testnetzwerk. Dies soll durch eine selbst entwickelte Simulationsumgebung realisiert werden. Hierbei soll jeder Knoten durch einen einzelnen Prozess repräsentiert werden. Beispiel für eine mögliche Topologie:



Anforderungen an das Netzwerk:

1. Das Netzwerk soll mindestens N (N >= 10) Knoten haben.
2. Das Netzwerk soll **mindestens 2*N und nicht mehr als 3*N Kanten** haben.
3. Jedem Knoten soll zugeordnet sein:
   - eine (eigene) IP-Adresse und Port
   - die IP-Adressen und Ports aller ihm direkt benachbarten Knoten
   - der auf dem Knoten verfügbare Speicherplatz (eine Zahl zwischen 1 und 999)

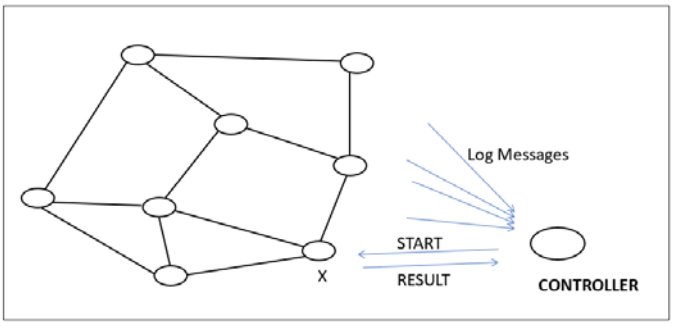
4. Das Netzwerk soll in einem JSON-File "**mynet.json**" spezifiziert werden, dessen Format Sie selbst festlegen können.

Anforderungen an die Simulation:

5. Jeder Knoten soll durch einen einzelnen **Prozess** simuliert werden.
6. Die für einen Knoten relevanten Werte (siehe 3.) sollen dem realisierenden Prozess über Kommandozeilenparameter (**args**) mitgegeben werden.
7. Eine Deployment-Komponente soll die Prozesse gemäß dem **"mynet.json"** starten
8. Die Übertragung von Nachrichten soll über UDP realisiert werden.
9. Es soll eine für jede Nachricht unterschiedliche, **zufällige Übertragungszeit (Latenz**) simuliert werden.
   (Hinweis: dies kann simuliert werden, indem nach jedem Receive-Vorgang ein sleep(w) mit einem Zufallswert 0 < w < 100 msec stattfindet.)

2.5.3 Teil 3: Start, Stop und Logging

Ein weiterer "CONTROLLER"-Prozess, soll die Summenbildung anstoßen und den Fortschritt des Algorithmus aufzeichnen.



Anforderungen an den Controller-Prozess:

1. Beim Startup des Controller Prozess soll dieser eine Übersicht des Netzwerks ausgeben, insbesondere:
   - Gesamtanzahl Knoten
   - Gesamtanzahl Kanten.
   (Hinweis: Er soll diese Werte dynamisch aus dem **mynet.json** ermitteln.)

2. Nach Startup fragt der CONTROLLER-Prozess über eine Benutzereingabe nach dem gewünschten Initiator-Knoten (Format IP-Adresse:Port) und schickt diesem eine START-Nachricht.

3. Nach Erhalt der START-Nachricht beginnt der Initiator-Knoten mit dem Algorithmus.

4. Während des Ablaufs, verschickt jeder Knoten im Netzwerk kontinuierlich, nach jedem Empfang einer Nachricht (nach dem sleep) eine LOG-Nachricht an den CONTROLLER mit folgenden Inhalten:

   a) Zeitstempel
   b) Start-IP/Port
   c) Ziel-IP/Port
   d) Typ der Nachricht (I oder E)
   e) ggf. Inhalte (nur E: Teilsumme).

5. Der Controller-Knoten gibt diese Nachrichten als fortlaufenden Log aus.

6. Nach Terminierung des Algorithmus schickt der Initiatorknoten eine RESULT-Nachricht mit der durch den Algorithmus berechneten Summe an den CONTROLLER.

7. Der Controller-Knoten gibt den Inhalt des RESULT als Gesamtsumme aus.

8. Zusätzlich gibt der Controller zum Schluss die Gesamtanzahl ausgetauschter I-Nachrichten und die Gesamtanzahl ausgetauschter E-Nachrichten aus.
   (Hinweis: Er zählt die eintreffenden Log-Nachrichten vom Typ I und E)

2.6 Labortermin 1: Tipps und Hinweise

- Die Deployment-Komponente kann Teil des Controllers sein, oder eine separate Komponente (die neben den Prozessen dann auch den Controller selbst startet).
- Nur der Controller Prozess braucht ein Display.
- Für Labortermin 1 und die Heimarbeit können Sie davon ausgehen, dass alle Prozesse auf einem einzelnen Entwicklungsrechner laufen. (aber bereits IP-Adresse + Port vorsehen!).
- Beginnen Sie bei Tests mit kleinen Konfigurationen (z.B. 3 Knoten) und vergrößern Sie dann die Knotenzahl. Implementieren Sie zunächst nur das Flooding, erst später den Echo-Algorithmus
- Zum Starten von lokalen Prozessen über Java-Code verwenden Sie am besten den `Java ProcessBuilder`. Lesen Sie das `mynet.json`, erzeugen Sie die dort spezifizieren Knoten/Prozesse und geben Sie die passenden Parameter (`args`) mit.

  Tipp: Damit Sie fürs Debugging anfänglich Output der generierten Prozesse sehen, verwenden Sie `processBuilder.inheritIO()` - Später brauchen Sie das nicht mehr.

- Sie müssen beim `ProcessBuilder` den Namen eines Executables angeben. Im Anhang finden Sie
  1. ein `build.gradle.kts` mit denen Ihnen IntelliJ ein Fat JAR-File baut, welches alle Bibliotheken beinhaltet (File ist auch auf Moodle zum Download)
  2. ein `startnode` Script-File, welches als Wrapper um das Jar dient, welches Sie im ProcessBuilder verwenden können (File ist auch auf Moodle zum Download)

2.7 Labortermin 2

Am Labortermin 2 steht wieder das Verteilte Testen im Vordergrund.

Sie sollen ein Netzwerk bestehend aus Prozessen aller teilnehmenden Gruppenmitglieder austesten, zum Beispiel:

- ein mynet.json, was insgesamt 10 Knoten auf 3 Rechnern startet
- 1 Controller und Deployment-Komponente von Studierendem X auf PC-A
- 3 Nodes mit Code von Studierendem 1 auf PC-B
- 4 Nodes mit Code von Studierendem 2 auf PC-C
- 3 Nodes mit Code von Studierendem 3 auf PC-D

Der Code soll passend vorinstalliert sein, das automatische Deployment soll von Rechner A ausgehend die Prozesse mit den jeweiligen Nodes auf Rechner B, C und D starten.

Ihre bisherige Deployment-Komponente soll so angepasst werden, dass sie das Remote Starten über SSH realisiert.

Anpassung der Deployment-Komponente und Testing sind **Gruppenarbeit** !

Empfohlene Vorgehensweise und Tipps

1. **Zum Starten von Prozessen auf Remote Knoten verwenden wir ssh.**
   Damit ssh funktioniert ist vorbereitend etwas Systemtechnik nötig :

SSH-Howto:

a) Führen Sie auf PC-A das Kommando "ssh-keygen" aus. (Drücken Sie dabei 3 mal Return bei Name und Passphrases um die Defaults zu übernehmen).
   → Es werden zwei Dateien id_rsa und id_rsa.pub (private key und public key) im /home/student/.ssh – Verzeichnis generiert.

b) Kopieren Sie das **`id_rsa.pub`** (public key) in das Home-Verzeichnis (/home/student) aller Rechner auf welchen remote ein Programm gestartet werden soll (PC-B/C/D). Dies kann über Moodle Upload/Download geschehen.

c) Aktivieren Sie den public key auf PC-B/C/D mit dem Kommando

```
cat >>/home/student/.ssh/authorized_keys </home/student/.ssh/id_rsa.pub
```

e) Denken Sie daran die Firewall für SSH auf allen Rechnern freizuschalten

```
firewall-cmd --add-port=22/tcp
```

Überprüfen Sie die korrekte Funktionsweise indem Sie von PC-A aus ein paar Kommandos auf den anderen Rechnern starten, z.B. mit

```
ssh student@xxx pwd
oder
ssh student@xxx ls
```

2. Was noch fehlt ist die Nutzung von ssh vom Programm aus, um basierend auf dem mynet.json flexibel Knoten starten zu können. Entwickeln Sie hierzu ein Java-Deployment Programm, welches ssh als Unterfunktion aufruft.

3. Eine Möglichkeit hierfür ist die `java.lang.Runtime exec`-Bibliotheksfunktion. Beispielcode:

```
List<String> commandList = new ArrayList<>();

commandList.add("ssh");
commandList.add("-o");
commandList.add("BatchMode=yes");
commandList.add("student@......");  //.. der jeweilige Knoten aus dem mynet.json
commandList.add("home/student/IdeaProjects/.../scripts/startnode"); //.. Ihr Projekt
commandList.add ("…");  // Ihre Parameter
commandList.add ("…")
…

String[] commandArray = commandList.toArray(new String[0]);

Process process = Runtime.getRuntime().exec(commandArray);
```

4. Falls Sie fürs Debugging den Output des Remote Scripts brauchen, können Sie verwenden:

```
  private static void streamAsync(String prefix, InputStream stream) {
      Thread t = new Thread(() -> {
          try (BufferedReader reader = new BufferedReader(new
                                              InputStreamReader(stream))) {
              String line;
              while ((line = reader.readLine()) != null) {
                  System.out.println(prefix + ": " + line);
              }
          } catch (IOException e) {
              System.err.println(prefix + ": " + e.getMessage());
          }
      });
      t.setDaemon(true);
      t.start();
  }
```

Aufruf dann nach dem `exec` mit:

```
streamAsync("..Ihre Knotenidentifkation… - STDOUT: ", process.getInputStream());
streamAsync("..Ihre Knotenidentifkation… - STDERR: ", process.getErrorStream());
```

5. **Tipp:** Zum Aufräumen können Sie `killall java` verwenden !

   von PC-A aus:

```
killall java && ssh student@IP-PCB killall java && ssh student@IP-PCC  killall java &&
                                                    ssh student@IP-PCD killall java;
```

   als kleines Script oder über Bash-History immer wieder holbar

Tipp: Zum Debuggen der ausgetauschten Nachrichten können Sie Wireshark verwenden.

Tipp: Denken Sie auch daran, bei remote Tests die Firewalls der beteiligten Rechner für Ihre Ports freizuschalten. (`firewall-cmd --add-port=…/udp`)

Heimarbeit

Als Heimarbeit sollen Sie ihre Lösung so umschreiben, dass der Initiator die Top-5 der Speicherplatzverbraucher ermittelt und ausgibt.

Die Lösung soll wieder für beliebige Topologien funktionieren. Die Nachrichtenlänge sei beschränkt.

Die Heimarbeit muss NICHT mehr in der Gruppe getestet werden. Ein korrekter Lauf auf dem eigenen Rechner reicht.

2.8 Erweiterungen (Optional)

Falls Sie noch Zeit haben, können Sie folgende Erweiterungen vornehmen:

1. Der Controller hat eine zusätzliche "RESET“-Funktion, welche eine RESET-Nachricht per Flooding an alle Netzknoten verteilt und diese veranlasst, einen neuen Eigenwert zu würfeln (rand()-Funktion). Jeder Knoten schickt unmittelbar nach dem Würfeln eine LOG-Nachricht mit dem gewählten Eigenwert (d.h. der angenommenen verfügbaren Speichergröße) an den Controller. Der Controller speichert sich diese Meldungen in einer Liste. Stößt der Benutzer jetzt die "SUM"-Funktion an, wartet der Controller bis er das Ergebnis erhält und überprüft es AUTOMATISCH auf Korrektheit (indem er die zwischengespeicherte Liste durchgeht und deren Summe ermittelt).

2. Erweitern Sie ihre Implementierung so, dass mehrere Berechnungen parallel durchgeführt werden können. D.h. der Controller stößt ein "SUM" für Rechner X und ein weiteres "SUM" für Rechner Y an. Beide berechnen die Summe gleichzeitig und melden diese zurück.

3. Implementieren Sie eine Funktion, die statt der Summe das MIMIMUM aller Knotenwerte ermittelt. Können Sie den Algorithmus so entwerfen, dass er GLEICHZEITIG für alle Knoten das Minimum berechnet. (d.h. am Ende des Algorithmus hat nicht nur der Initiator sondern jeder Knoten in einer zusätzlichen Variable das Minimum gespeichert)?

Dieser einfache Algorithmus lässt sich ideal kombinieren mit weiteren verteilten Algorithmen, die wir in der Vorlesung erst gegen Ende des Semesters kennenlernen werden (Snapshot-Algorithmen, Terminierungsalgorithmen). Es steht Ihnen frei, diese nach Semesterende noch selbständig im gegebenen Testumfeld einzusetzen. Den Testrahmen dafür haben Sie bereits entwickelt, damit ist die Implementierung neuer Elemente technisch relativ einfach.

2.9 Abgabe (Post-Lab)

Labortermin 1

Zwei Wochen nach dem ersten Labortermin bitte 1 ZIP-File mit dem folgenden Inhalt hochladen:

1. mynet.json
   Hinweis: in der Abgabeversion müssen die Knoten die Storage-Werte 1, 2, 3, .. N haben
2. Source Code für Node und Controller (einschließlich Deployment)
3. Executables
4. Gesamte Ausgabe des Controllers
   (Beachten Sie: **N >=10; 2N <= Kantenzahl <= 3N**,
   Inhalte der Ausgabe müssen alle Anforderungen von Seite 10 erfüllen)

Labortermin 2

Ein ZIP-File mit Lösung der **Heimarbeit**:

1. mynet.json (wie oben)
2. Source Code für Node und Controller
3. Source Code für das ssh-basierte Deployment
   (Start der remote Prozesse gemäß mynet.json)
4. Executables
5. Gesamte Ausgabe des Controllers
   (Beachten Sie: N >=10; 2N <= Kantenzahl <= 3N,
   Inhalte der Ausgabe müssen alle Anforderungen von Seite 10 und der Zusatzaufgabe von Seite 14 erfüllen)
6. Der Start der Prozesse in Form eines Deployment Log (Ausgabe der ssh-Aufrufe einschließlich aller Parameter; durch Trace-Ausgabe des Controllers/Deployers)

Appendix: Template für Gradle-Build File `build.gradle.kts` (bitte anpassen: package, java version, ..)

File ist auch in Moodle !

```
plugins {
    java
    application
}
group = "org.example"
version = "1.0.0"

repositories {
    mavenCentral()
}
dependencies {
    implementation("com.google.code.gson:gson:2.11.0")
}
java {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
}
tasks.withType<JavaCompile>().configureEach {
    options.encoding = "UTF-8"
    // Erzeugt Java-17-kompatible Klassen, wenn mit JDK 17 oder neuer kompiliert wird.
    options.release.set(17)
}
application {
    mainClass.set("org.example.lab3.MyController")
}
// Fat-Jar, damit die Skripte ohne Gradle-Aufruf funktionieren.
val fatJar by tasks.registering(Jar::class) {
    archiveFileName.set("vs-lab3.jar")
    duplicatesStrategy = DuplicatesStrategy.EXCLUDE
    manifest {
        attributes["Main-Class"] = "org.example.lab3.MyNode"
    }
    from(sourceSets.main.get().output)
    dependsOn(configurations.runtimeClasspath)
    from({
        configurations.runtimeClasspath.get().map { dependency ->
            if (dependency.isDirectory) dependency else zipTree(dependency)
        }
    })
}
tasks.named("build") {
    dependsOn(fatJar)
}
```

Appendix: HOWTO - Wie verpacke ich JAR in ein Script (wichtig für SSH-Zugriff Template für Gradle- Build File

(bitte anpassen: project, package, main ..)

File ist auch in Moodle !

1. Im Projektverzeichnis `scripts`-Directory anlegen und dort File anlegen, z.B. `startnode`
2. `startnode` editieren und mit Inhalt der folgenden Art füllen:

```
#!/usr/bin/env bash
java -cp ~/IdeaProjects/<name_of_proj>/build/libs/<name_of_jar>.jar \
                                        “org.example.<name_of_main>” "$@"
```

3. Danach `startnode` ausführbar machen mit

```
chmod 775 ~/IdeaProjects/scripts/startnode
```

4. Sollte jetzt im IntelliJ Terminal startbar sein mit

```
~/IdeaProjects/scripts/startnode
```